Implementation Plan — AI Legal Assistant for Indian Laws

Deadline: March 29, 11:59 PM IST | Duration: 8 days

Day 1–2 (Mar 22–23) — Foundation & Data Setup

Goal: Get the environment ready and legal data indexed.

- Create a public GitHub repo. Set up proper folder structure: /data, /agents, /ui, /logs. Write a basic README immediately so commit history starts clean.
- Install the Python stack: langchain, fastapi, faiss-cpu, sentence-transformers, pymupdf, pdfplumber, streamlit.
- Download the legal corpus — all are freely available in the public domain: Indian Constitution, IPC (Indian Penal Code), CrPC (Code of Criminal Procedure), CPC (Civil Procedure Code), Consumer Protection Act 2019.
- Parse these PDFs into text chunks (400–500 tokens per chunk with 50-token overlap). Generate sentence embeddings using all-MiniLM-L6-v2 from HuggingFace. Save the FAISS index to disk.

Output by end of Day 2: A working FAISS index you can query and get back relevant legal text chunks.

Day 3 (Mar 24) — Build the Three Sub-Agents

Goal: Each agent works independently before connecting them.

- **Retrieval agent:** LangChain RetrievalQA chain connected to your FAISS index. Takes a query, returns top-3 relevant chunks with source document name and section number.
- **Reasoning agent:** LangChain LLMChain with a carefully written chain-of-thought prompt. The prompt must instruct the model to: state which law applies, identify the user's specific situation, list the legal steps, and flag if information is missing.
- **Compliance agent:** Rule-based first, then LLM-assisted. Hardcode triggers for: criminal matters (FIR, arrest, bail), urgent situations (domestic violence, eviction notices), and out-of-scope queries (tax law, corporate law). These trigger a "please consult a licensed lawyer" disclaimer automatically.

Output by end of Day 3: Three independently testable agent functions.

Day 4 (Mar 25) — Build the Orchestrator + Audit Logger

Goal: Connect all agents and make every decision traceable.

- **Orchestrator:** Use LangChain AgentExecutor. It receives the processed query, calls the retrieval agent first, passes the retrieved chunks to the reasoning agent, then runs the compliance agent check on the final answer before it is returned to the user.
- **Audit logger:** Every time an agent fires, write a JSON entry to a log file containing: timestamp, query text, which agent was called, which document chunks were retrieved (with chunk IDs), reasoning chain summary, and whether compliance triggered. This is critical for the PS5 "auditability" criterion.
- **Error handling:** If the retrieval agent returns zero results, the orchestrator must return a graceful fallback — "I couldn't find relevant Indian law for this query. Could you provide more details?" — not a crash or hallucinated answer.

Output by end of Day 4: Full agent pipeline working end-to-end in a Python script, with audit logs being written correctly.

Day 5 (Mar 26) — UI + Multilingual Layer

Goal: Make it usable and demo-ready.

- Build a Streamlit chat interface. Display: user message, AI response, source citations (which act, which section), and a collapsible "How I answered this" panel showing the audit trail for that specific query.
- Integrate IndicTrans2 (by AI4Bharat, free and open-source) for Hindi output at minimum. The pipeline becomes: answer in English → translate to user's selected language → display both.
- Test three complete end-to-end scenarios that you will demo: (1) tenant-landlord dispute, (2) consumer product complaint, (3) workplace harassment query.

Output by end of Day 5: A working UI your group can actually click through.

Day 6 (Mar 27) — Edge Case Testing + Audit Viewer

Goal: Harden the system and prove it handles tricky inputs.

- Test deliberately bad inputs: extremely vague queries ("I have a legal problem"), queries spanning two states with different rules, queries missing key facts (no date, no location). Verify the system asks clarifying questions rather than guessing.
- Test compliance triggers: type "I was arrested yesterday" — verify the compliance agent fires and the lawyer disclaimer appears. Type "I want to file a murder case" — verify it routes to "consult a lawyer" immediately.
- Polish the audit trail viewer in the UI. Judges will look at this closely — make sure it clearly shows the decision path for every query, not just a raw JSON dump.

Output by end of Day 6: System handles at least 10 edge cases gracefully, with clean audit logs for all of them.

Day 7 (Mar 28) — Documentation + Video

Goal: Finish all four submission deliverables except the final code push.

- **Architecture document (1–2 pages):** Include the architecture diagram (already done), one paragraph per layer describing what it does, the tech stack table, and the PS5 criteria mapping. Keep it under 2 pages.
- **Impact model:** Use these numbers — India has 14 crore+ demat accounts and 140 crore+ citizens. A basic lawyer consultation costs ₹1,500–5,000 per visit. If the system handles 10,000 queries/month at ₹0 cost, that is ₹1.5–5 crore in legal access value per month. State your assumptions clearly.
- **3-minute pitch video:** Structure it as — 45 sec on the problem (legal inaccessibility in India), 90 sec on live demo (run all three test scenarios), 30 sec on architecture overview, 15 sec on impact numbers. Record in one clean take, no editing needed.

Output by end of Day 7: All documents ready, video recorded.

Day 8 (Mar 29) — Final Review & Submit

- Run the complete demo flow one final time on a fresh machine to confirm nothing is broken.
- Verify the GitHub repo has: all source code, clean README with step-by-step setup instructions, and at least 20–30 commits showing the build process (not one giant commit at the end).

- Submit before 11:59 PM IST. Do not wait until the last hour.

Quick Tech Stack Summary for the Group

What	Tool
Agent framework	LangChain
Vector database	FAISS (local, free)
Embeddings	all-MiniLM-L6-v2 (HuggingFace)
LLM	Claude API or GPT-4o
UI	Streamlit
Multilingual	IndicTrans2 (AI4Bharat)
PDF parsing	PyMuPDF + pdfplumber
Backend API	FastAPI
Audit logging	Python logging → JSON file